

Interpreting Regulation with Accountable Ensembles

From SFC circulars to competing interpretations,
bounded resolution and verified Java controls

LegalMath

A technical account for compliance, legal, engineering and model-risk readers. The running case is SFC circular 23EC46 and a retained, deliberately unapproved implementation of its selected gift restriction. The proposed ensemble makes agreement, disagreement, missing evidence and the limits of automatic resolution visible before a control reaches a bank system.

22 September 2026

Complete author draft, version 1.0

Public-source and synthetic examples. Independent compliance, technical and reader acceptance remain pending. Proposed extensions are distinguished from the currently implemented workbench throughout.

The decision this book supports

A private bank cannot treat a software test result as a complete answer to a regulatory question. A program may reproduce a decision table exactly while the table overlooks an exception, applies to the wrong activity, or relies on a term whose meaning has never been established. The practical question is how to make those mistakes harder to introduce, easier to discover, and impossible to hide inside a confident-looking automated answer.

This book develops an interpretation workbench that retains several candidate readings, investigates the reasons for and against them, checks their consequences, and keeps unresolved questions attached to the control. Multiple methods provide different opportunities to discover error. A discrepancy opens a recorded resolution process with finite limits. Reaching those limits produces a report of the remaining uncertainty and prevents release of an affected control; it does not cause the system to choose the most popular answer.

The intended reader need not know a formal legal language, an argumentation semantics or a tree-search algorithm. Those methods are developed through a single circular and deliberately contrasting client scenarios. Engineers should be able to derive data structures, state transitions and acceptance tests from the text. Compliance readers should be able to challenge the source mappings, scope, classifications and review decisions without reading the software.

There are two distinct contributions. The first is an explanation of what the literature and the existing LegalMath evidence establish. The second is a new engineering design that combines those methods. The design is a proposal for implementation and evaluation; published results about another task do not establish its performance on SFC circulars. Finite search also cannot establish that every defensible legal reading has been found. The design instead makes coverage obligations, source dependencies, disagreement and stopping reasons explicit, and assigns responsibility for the decisions automation cannot settle.

The ten chapters follow the order in which a bank would have to justify a control. The initial circular exercise exposes the gap between execution and interpretation. Formal meaning and relevant languages then make the alternatives precise. Legal argumentation and search preserve competing readings. The central ensemble chapter specifies their bounded investigation. Verification, Java integration, service contracts, evaluation and continuing operation complete the route from a source document to an attributable bank decision.

The companion machine-readable contracts describe a proposed extension, not a silent alteration of the accepted runtime. The existing oracle and source snapshots are preserved. A successful document build establishes that this manuscript compiles; it does not supply the independent legal judgment, empirical validation or reader feedback required to adopt the product.

Contents

The decision this book supports	i
1 A correct program can enforce the wrong rule	1
1.1 The decision before the decision	1
1.2 What the existing experiment actually established	2
1.3 Five places where meaning can be lost	2
1.4 The denominator problem in completeness	3
1.5 Why agreement is useful and insufficient	5
1.6 The object being classified is part of the rule	6
1.7 Time changes the question being answered	6
1.8 From an answer to an accountable decision	7

Chapter 1

A correct program can enforce the wrong rule

1.1 The decision before the decision

A distributor proposes a marketing offer involving an investment fund. The offer includes an incentive. A compliance control must determine whether the incentive falls within a restriction, whether an exception applies, and whether the relevant evidence is sufficient to make that determination. A software team can make these questions look simple: gather a few fields, evaluate a condition, and return a result to the bank's Java system. The difficult decisions have already occurred before the condition runs. Someone has decided what counts as an incentive component, what the words in the circular cover, what evidence establishes a promotional link, and which version of a referenced rule applies.

The distinction is consequential. Suppose a gift restriction covers both a named investment product and a type of investment product. An implementation that checks only the first route may work perfectly for a demonstration using one named fund. Its unit tests may be comprehensive with respect to the code that was written. Its developer may correctly explain every branch. The defect is that a branch required by the adopted interpretation never reached the code. Testing generated branches alone will not reveal a missing source obligation.

Now suppose that both promotional routes are represented, but an exception for a fee discount is applied to an entire offer containing a fee waiver and a separate voucher. The formal expression can be well typed, consistent and executable. Its output can be reproduced by Python, Java and a solver. The unresolved question is the object to which the exception attaches. A Boolean variable named `discount` conceals that question unless its definition identifies the assessed component and the evidence supporting the classification.

These are hypothetical failure scenarios derived from the retained 23EC46 exercise, not findings about a deployed bank system. They explain why the user regards interpretation error as a material operational risk. Regulatory action, reputational damage and accountability for an erroneous control make the quality of the decision process important. This book does not estimate the probability or financial severity of an enforcement outcome. It specifies the evidence a bank would need to understand, challenge and authorize its own interpretation.

The system we want to build therefore begins one step earlier than conventional rule implementation. It asks what the bank has grounds to encode, which alternative readings have been considered, what remains unknown, and who has authority to resolve the uncertainty. Code generation follows that record. It does not substitute for it.

The unit of work is a *regulatory change assessment*. It includes source material, a declared business scope, proposed interpretations, affected controls, data requirements, implementation changes and review decisions. A single circular can create several such assessments because different provisions may concern different actors or activities. Conversely, one assessment may need several circulars, Code provisions, FAQs and earlier decisions. Treating one downloaded page as a self-contained legal specification would lose these relationships.

1.2 What the existing experiment actually established

The present LegalMath workbench provides a useful starting point because it has already crossed the engineering boundary from a formal rule to generated Java. Its second-circular exercise concerns a deliberately narrow interpretation of paragraph 10 of SFC circular 23EC46, dated 24 October 2023 (SFC, 2023). The example assesses one incentive component in a specified distributor and SFC-authorized-fund profile. Whether the component is a gift, whether it is a fee discount, and whether it has a relevant product link are supplied classifications. The program does not establish those classifications by reading an offer document.

The exercise preserved a packet of 22 named expected outcomes before constructing the candidate rule. A separate truth-table implementation enumerated 729 combinations of true, false and unknown values across six inputs. The actual generated Java class and the Python evaluator agreed on the declared results. Four deliberately altered rules were compiled and detected by the named cases. Those are substantial engineering observations: they show that the exercised implementation computes the proposed rule in the checked domain, and that the cases can expose several particular errors.

They do not provide independent evidence that the proposed rule is the correct reading of the circular. The same implementing agent authored the interpretation and the expected outcomes. Writing one before the other makes later alteration detectable; it does not create a second legal judgment. The separate truth-table oracle removes a direct dependency on the candidate's abstract syntax tree. It still expresses the same chosen formula. If that choice omitted a legal condition, all three implementations could agree on the omission.

The candidate also retained five unresolved review issues and two unresolved source dependencies. Its attempt to enter review was rejected. It remained a draft and no release was created. That refusal is part of the useful result. The workbench did not mistake successful compilation for authority to deploy a regulatory control. The monograph preserves this example as a record of an unapproved candidate; resolving the cited dependencies here by assumption would destroy the very distinction the example teaches.

The broader application acceptance contains 154 passing tests, but its drafting provider returns supplied fixtures. It has not interpreted arbitrary circulars in a measured live-model evaluation. The acceptance count is evidence about source intake, data contracts, execution, review workflow and delivery under the tested conditions. It is not an estimate of translation accuracy. A dashboard that presents the count without the target of the tests would invite a stronger conclusion than the evidence supports.

This distinction suggests the question the new product must answer. Given a source packet and a proposed operational scope, can it expose a material misinterpretation before that misinterpretation becomes an approved executable control? Success requires more than generating valid syntax. It requires finding missing alternatives, retrieving the context that changes a reading, and making disagreements concrete enough for a competent reviewer to resolve.

1.3 Five places where meaning can be lost

It is tempting to describe the workflow as one translation from English to Java. That description hides several different kinds of error. The first is source error: the reviewed text is incomplete, extracted incorrectly, superseded, or wrongly treated as authoritative. An omitted footnote can

remove a definition. A copied paragraph can lose a heading that limits its scope. A source's publication date can be mistaken for its effective date. None of these failures requires a language model to reason incorrectly; the model may never receive the material needed for the correct decision.

The second is interpretation error. The words are present, but their legal function is misrepresented. A qualification becomes an unconditional rule. An exception is attached to the wrong object. A statement about possible classification becomes a finding of fact. A normative expectation is converted into an operational block without recording the bank's choice. These are decisions about meaning, not merely extraction or programming defects.

The third is specification error. The reviewer may have a defensible interpretation, but the intermediate rule expresses something else. A conjunction is replaced by a disjunction, a threshold loses equality, or an unknown condition is interpreted as false. The specification can also fail through insufficient expressiveness: a time-dependent obligation may be forced into a one-off Boolean test. A software language that cannot represent a required distinction must report the gap rather than approximate it silently.

The fourth is execution error. Java can evaluate a correct intermediate rule incorrectly because of a compiler defect, numeric conversion, time-zone mismatch, or inconsistent evidence handling. A data adapter may supply dollars where the rule expects cents. A build may package an older runtime. An apparent match on a few simple examples can miss a defect in unknown-value or conflict precedence. These failures are often amenable to strong formal or exhaustive checks once the specification and input domain are fixed.

The fifth is operational error. The correct Java result can be used for the wrong client, product, source version or transaction time. Consent can change between evaluation and order commitment. A system can treat the absence of one prohibition as permission to proceed despite unresolved checks elsewhere. A reviewed rule can remain active after its source is replaced. These are failures of integration and change control, even when the individual rule evaluator is correct.

Figure 1.1 gives each transition its own question. The arrows do not imply that success flows automatically from left to right. Every arrow needs evidence appropriate to the claim it makes.

A useful consequence follows immediately. Two checks count as meaningful redundancy only to the extent that their opportunities for failure differ. A second compiler can challenge code generation. It cannot detect a missing source document if it receives the same incomplete specification. A second language model may propose a different reading, but it cannot authenticate a source merely by describing it confidently. Redundancy must be designed around the failure being investigated.

1.4 The denominator problem in completeness

Suppose a drafting agent extracts eight requirements from a circular and produces eight rules. A coverage report then shows that every extracted requirement has a rule, every rule has a test, and every test passes. The result sounds complete. It is complete only relative to the agent's own extraction. If the circular contains a ninth operative requirement, the report has no row on which that omission can appear. The system has used its answer to define the denominator of its completeness measure.

The source inventory must therefore originate independently of the proposed rule set. At the

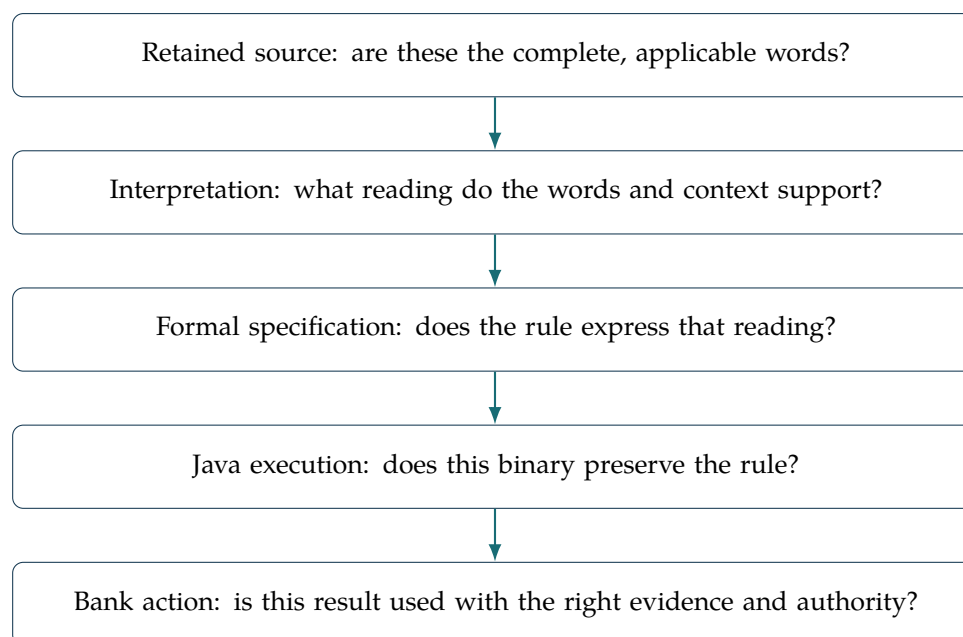


Figure 1.1: A correct downstream step does not repair a mistaken upstream premise. Each transition has a different assurance question.

document level it records paragraphs, headings, annexes, footnotes, tables and incorporated references. At the normative level it records potential duties, prohibitions, permissions, definitions, exceptions and qualifications. These two inventories serve different purposes. A paragraph inventory can establish that a passage was considered, but one paragraph may contain several operative propositions. A rule inventory can establish that an identified proposition has a disposition, but it cannot prove that the propositions were all identified.

For the retained circular, the existing inventory contains 18 numbered paragraphs and four footnotes. Only one paragraph has an executable control in the second-circular example. Other passages concern context, other controls, qualitative assessment or external dependencies. It would be wrong to describe the circular as implemented because its selected gift rule passes 729 input combinations. It would also be wrong to implement every paragraph mechanically: an observation about market practices is not necessarily a new obligation, and a qualitative duty may need a reviewed procedure rather than a fabricated numeric threshold.

A complete disposition process gives every inventoried proposition a reasoned destination. It may be linked to an executable rule, to a reviewed manual procedure, to a justified exclusion from the declared assessment, or to an open question. An open question is visible incompleteness. An exclusion requires a scope rationale and reviewer authority. A manual procedure needs an owner and evidence of performance; attaching the word “manual” to an inconvenient clause does not implement it.

The same denominator problem occurs in search. If five candidate interpretations agree, the correct conclusion is agreement among those five candidates under their recorded assumptions. The system has not measured the interpretations it failed to generate. Breadth constraints and targeted challenges improve the opportunity to find missing readings, but there is no finite list of prompts that proves unrestricted legal interpretation has been exhausted. The product must report which dimensions were explored and which remain outside the search.

This makes completeness a collection of bounded claims. We can check that every retained para-

graph has a disposition. We can check that every cited reference has been resolved or explicitly left open. We can check that each represented exception has positive, negative and uncertain cases. We cannot infer from those facts that no relevant circular exists elsewhere or that every legally defensible reading has been found. A bank needs a process for setting and maintaining its regulatory perimeter as well as a translator inside that perimeter.

1.5 Why agreement is useful and insufficient

An ensemble is a group of methods whose results are examined together. In a prediction task it is common to combine answers by majority vote or averaging. For regulatory interpretation, the useful unit of combination is often a reasoned claim: one method identifies a scope restriction, another locates an exception, and another generates a case that exposes a missing promotional route. Their outputs need not be identical to contribute to the same assessment.

Agreement can provide evidence that a representation is stable across some transformations. A controlled-English sentence and its formal rule may produce the same decisions. A back-translation may preserve the formal structure. A second implementation may agree with the first. Each observation rules out some failure explanations. None rules out every explanation. In particular, methods that share the same omitted premise can agree for the same wrong reason.

Disagreement is therefore not merely a nuisance to suppress. It may be the first observable sign of a missing definition, a conflict in evidence, or a genuine interpretive choice. If one candidate treats a product-type promotion as covered and another does not, the difference points to a specific source phrase. If two candidates disagree only when a classification is unknown, the problem may be their evaluation semantics. If two sources have different effective intervals, the disagreement may be temporal rather than substantive. The controller should classify these possibilities before attempting a repair.

The converse is equally important: disagreement alone does not make every candidate legally respectable. A generated rule that deletes an express exception may be a useful negative control but a poor interpretation. The system should retain it long enough to explain the rejection and test the detectors, while distinguishing it from a reading that remains genuinely unresolved. A shortlist filled with contrived alternatives can increase reviewer workload without improving assurance.

The proposed ensemble consequently has two obligations. It must preserve material dissent until there is a recorded reason to resolve it, and it must identify the evidentiary standing of that dissent. An objection supported by an applicable source is different from an unsupported model suggestion. A solver counterexample proves a difference in formal consequences; it does not by itself give one interpretation greater legal authority. A human reviewer can accept a reading for a declared scope, but that decision must not be mislabelled as a machine proof of English meaning.

Chapter ?? will specify how these distinctions become a finite automatic process. The core idea is that a discrepancy creates work with a defined purpose: retrieve missing authority, correct an encoding, investigate an alternative or clarify a fact. Repetition without a new diagnostic objective is not additional assurance. When the permitted actions have been exhausted, the system must make the remaining issue easier for a reviewer to understand.

1.6 The object being classified is part of the rule

The incentive example reveals a general weakness of compact decision tables: they often define a field without defining the entity to which it belongs. A field called `is_discount` could describe a payment, a fee line, a marketing offer, a product or a client relationship. These objects are not interchangeable. If a fee waiver and voucher are separate incentive components, a property of the waiver cannot automatically be transferred to the voucher. The rule's subject identity is therefore part of its meaning.

The source-to-rule specification must state the unit of assessment before introducing its predicates. In this example it is one proposed component within a documented offer. The specification must also state how components are identified and when a purported separation is legally meaningful. A data designer cannot settle that second question solely by assigning two database identifiers. Operational decomposition needs to reflect the reviewed legal analysis; otherwise a bank could accidentally change the meaning of an exception by changing how it stores an offer.

The same problem appears in private-bank suitability workflows. A statement about a client's status does not necessarily apply to every account, every product category or every transaction channel. Consent may be linked to an identified service and period. A relationship manager's assessment may have a subject, scope and evidence date that differ from those of a wealth record. The intermediate representation must retain these bindings, not merely a collection of client-level Boolean fields.

A useful specification sentence therefore contains both a predicate and its subject: "component C in offer O is classified as a fee discount under definition D, on the evidence recorded in assessment A." That sentence is longer than "discount = true," but the extra information explains what the Boolean means. The eventual runtime can carry compact identifiers so that ordinary transactions remain efficient. The review record retains the definitions and evidence those identifiers name.

This is also why a language model's ability to extract entities is insufficient. It may correctly identify the words "fee waiver" and "voucher" while failing to determine the legal relationship between them. Extraction can propose candidates for classification. The system must separately record the reasoning that accepts their relationship and the uncertainty that survives it. That separation lets a later source clarification update a definition without pretending that the underlying marketing document changed.

1.7 Time changes the question being answered

An apparently straightforward rule can yield different legitimate assessments at different times. The regulation may change, the facts may change, or the bank may learn something new about an earlier event. These are three distinct sources of variation. A source publication date identifies when a document was issued. An effective interval identifies when a reviewed rule is intended to apply. An evidence recording time identifies when the bank had the relevant information. Substituting one date for another changes the assessment.

Consider a gift classification entered on Tuesday for an offer made on Monday. A historical review can ask what the bank should conclude now about Monday's offer, using Tuesday's evidence. It can also ask what the system could conclude on Monday from the information

then recorded. Both questions are useful. They require different knowledge cutoffs. Replaying the earlier decision with all subsequently available facts would conceal the information gap that existed at the time.

The current runtime makes assessment time and knowledge cutoff explicit through `valid_at` and `known_at`. Known evidence has a validity interval and a recording timestamp. An expired classification becomes unknown for an assessment outside its interval. Evidence recorded after the historical cutoff cannot be used in that historical assessment. A superseding record names the record it corrects; simple arrival order does not determine which assertion is authoritative.

These are engineering conventions selected for a precise replay question. They do not determine the legal effect of every late discovery or correction. A corrected factual record may establish that an obligation was performed on time, while the bank's earlier inability to demonstrate that fact remains part of the audit history. A regulatory amendment may apply prospectively, retrospectively or with transition rules that need interpretation. The translator must represent the relevant temporal question before the runtime can answer it.

The ensemble must therefore challenge time assumptions as well as the visible substantive condition. One candidate can be correct under a historical source version and wrong under the current one. Another can use a current definition to interpret an earlier transaction without justification. A disagreement report that omits the source revision and assessment times can make these distinct questions look like a conflict about the same rule. Immutable context identifiers are necessary to know what is actually being compared.

1.8 From an answer to an accountable decision

A useful product result contains more than a sentence announcing compliance or non-compliance. It identifies the precise condition evaluated, the reviewed scope, the facts and definitions used, the source version, the proposed interpretation, the evidence of correct execution, and any unresolved dependency. These details serve different readers. A compliance reviewer needs to see the interpretation. A data owner needs to see the evidence mapping. An engineer needs reproducible inputs and an artifact identity. A release owner needs to know which approvals bind the exact binary being deployed.

The design should not force every reader to inspect every detail on every case. The review interface can begin with the decision and the material reason, then allow inspection of assumptions and counterexamples. The underlying record must still support that inspection. A concise explanation is valuable when it is a faithful view of a fuller record; it is dangerous when it replaces a missing record with persuasive prose.

Accountability also requires distinguishing an interpretation from a bank policy. A bank may decide to prohibit a class of offers while a regulatory question remains unresolved. That can be a sensible operational choice under its own authority. The system should record the policy owner, scope and reason for the restriction. It should not attribute the restriction to an SFC sentence unless the interpretation supports that attribution. Otherwise later reviewers cannot tell whether a change requires new legal analysis or a policy decision by the bank.

Likewise, a manual review route must have a defined effect. "Refer to compliance" is incomplete if no owner, service expectation, required evidence or transaction state is specified. An unresolved assessment may hold an automated approval, route the case for review or invoke an already approved conservative policy. The choice belongs to the bank's operating design. The

translator should never invent a transaction permission merely because its retries have reached a limit.

The product's ambition is thus substantial but specific. It should improve the quality of the questions reaching a human reviewer, preserve alternatives that would otherwise disappear, and verify execution of the interpretation that is eventually adopted. The next chapter follows the second circular through that entire route. Its apparent simplicity is useful: a small formula is sufficient to expose why independent source judgment, explicit scope and uncertainty cannot be replaced by a large number of successful program evaluations.

Bibliography

SFC. Circular on distributors providing additional returns and other services or arrangements when marketing SFC-authorised funds. 24 October, reference 23EC46, 2023. URL <https://apps.sfc.hk/edistributionWeb/gateway/EN/circular/doc?refNo=23EC46>.